Project Report — Austrian Tax Law Q&A

Author: Fedor Samorokov (h12406664)

---

1. Models Overview

Three models were built to answer 643 Austrian tax law questions (KStG 1988, EStG 1988, UStG 1994). Only one model (Model 3) uses an external API.

	Model 1 (Baseline)	Model 2 (Fine-tuned)	Model 3 (RAG)
Base model	`dbmdz/german-gpt2`	`dbmdz/german-gpt2`	`gpt-4o-mini` (OpenAI)
Parameters	124M	124M	proprietary
Architecture	GPT-2 (causal LM)	GPT-2 (causal LM)	Transformer decoder
Pre-training data	German web text (Wikipedia, news, etc.)	German web text	Proprietary (multilingual)
Approach	Direct inference	Supervised fine-tuning	Retrieval-Augmented Generation
API used	No	No	Yes (OpenAI)

---

2. Model Details

Model 1: Baseline Inference (dbmdz/german-gpt2)

A pre-trained German GPT-2 model used without any fine-tuning or domain adaptation. Each question is formatted as `"Frage: {question}\nAntwort:"` and the model generates a continuation.

Hyper-parameters: - `max_new_tokens`: 150 - `min_new_tokens`: 30 - `do_sample`: False (greedy decoding) - `no_repeat_ngram_size`: 3 - `pad_token_id`: eos_token_id

This model serves as a **baseline** to demonstrate what a general-purpose German language model produces without domain-specific training. Since GPT-2 was never trained on Austrian tax law, answers are expected to be generic and often incorrect.

Model 2: Fine-tuned GPT-2

The same `dbmdz/german-gpt2` model, fine-tuned on 152 Austrian tax law Q&A pairs using HuggingFace `Trainer` with causal language modeling (next-token prediction).

Training data: 152 Q&A pairs (`training_data.csv`) written from the actual law texts — KStG 1988, EStG 1988, UStG 1994. No overlap with the 643 test questions. Training data was generated with AI assistance from the law texts.

Fine-tuning hyper-parameters: - `num_train_epochs`: 3 - `per_device_train_batch_size`: 8 - `warmup_steps`: 100 - `weight_decay`: 0.01 - `max_length` (tokenization): 256 - Loss function: Cross-entropy (causal LM) - Optimizer: AdamW (default) - Platform: Google Colab (T4 GPU)

Inference hyper-parameters: - `max_new_tokens`: 150 - `do_sample`: False (greedy decoding) - `pad_token_id`: eos_token_id

Model 3: RAG with GPT-4o-mini

Retrieval-Augmented Generation using the actual Austrian tax law PDFs as source documents.

Retrieval model: OpenAI `text-embedding-3-small` (1536 dimensions)

Documents indexed: - KStG 1988 (Fassung vom 03.04.2026) — 209,147 characters - EStG 1988 — 927,140 characters - UStG 1994 — 306,123 characters

Preprocessing/Chunking: - Text extracted from PDFs using `pdfplumber` - Split by paragraph markers (§) using regex: `re.split(r'(?=§\s*\d+)', text)` - Minimum chunk length: 100 characters - Total chunks: 2,953 (468 KStG + 2,060 EStG + 425 UStG)

Retrieval: For each question, the top `k=3` most relevant chunks are retrieved using cosine similarity between the question embedding and chunk embeddings.

Generation hyper-parameters: - Model: `gpt-4o-mini` - `max_completion_tokens`: 300 - `temperature`: 0 - System prompt instructs the model to answer in German, cite relevant paragraphs, and respond in 1-3 sentences

---

3. Evaluation Methodology

Model outputs are evaluated against **ground-truth answers** from the Austrian Tax Law Dataset (`Austrian Tax Law Dataset - Dataset.csv`), which contains student-written `correct_answer` entries for each of the 643 test questions (dataset generated in our first assignment).

Metrics used:

1. **Exact Match** — strict string equality after normalization. Very strict: even a correct answer worded differently scores 0.
2. **BLEU-4** — n-gram precision with smoothing. Measures how many 1- to 4-grams in the prediction appear in the reference.
3. **ROUGE-1 / ROUGE-2 / ROUGE-L** — recall-oriented n-gram overlap. ROUGE-1 = unigram, ROUGE-2 = bigram, ROUGE-L = longest common subsequence F1.
4. **BERTScore** — semantic similarity using contextual embeddings from multilingual BERT (`lang=de`). Captures meaning beyond surface n-gram overlap — important for German legal text where correct answers can be paraphrased.

Reference-free quality metrics (no ground truth needed): - **Average word count** — answer completeness - **Trigram uniqueness** — ratio of unique trigrams to total (detects repetition; 1.0 = no repetition) - **Vocabulary diversity** — unique words / total words

The evaluation script is in `evaluation/model_evaluation.ipynb`.

---

4. Results

Main Results Table

All metrics are computed against ground-truth answers from the Austrian Tax Law Dataset.

Model	Exact Match	BLEU-4	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore F1
Model 1 (Baseline GPT-2)	0.0000	0.0048	0.1136	0.0170	0.0813	0.6294
Model 2 (Fine-tuned GPT-2)	0.0000	0.0037	0.1248	0.0123	0.0652	0.6533
Model 3 (RAG + GPT-4o-mini)	**0.0000**	**0.0518**	**0.3078**	**0.1233**	**0.2156**	**0.7395**

Model 3 (RAG) outperforms both other models across all metrics, as expected — it has access to the actual law texts at query time and uses GPT-4o-mini for generation. Models 1 and 2 rely solely on what `dbmdz/german-gpt2` learned during pre-training / fine-tuning.

Notably, Model 2 scores slightly lower than Model 1 on BLEU-4 (0.0037 vs 0.0048) and ROUGE-L (0.0652 vs 0.0813), despite fine-tuning. Fine-tuning on 152 examples shifted the model's vocabulary toward tax-related terms, but also caused it to hallucinate confident-sounding legal text that diverges from the ground-truth wording — hurting precision-based metrics. Model 2 does improve on ROUGE-1 and BERTScore, suggesting it picks up some topical relevance from fine-tuning even though surface-level precision degrades.

---

5. Error Analysis

Model 1 (Baseline GPT-2)

- **Main issue: Generic, off-topic responses.** The model was never trained on tax law, so it produces general German text that often has no relevance to the question.
- Answers tend to be long (~106 words) but say very little of substance (min number of tokens was used because otherwise the model produced only 1 word – german articles Die, Der, Das. . .)
- With greedy decoding + `no_repeat_ngram_size`, the model avoids pure repetition but still produces vague, circular text.

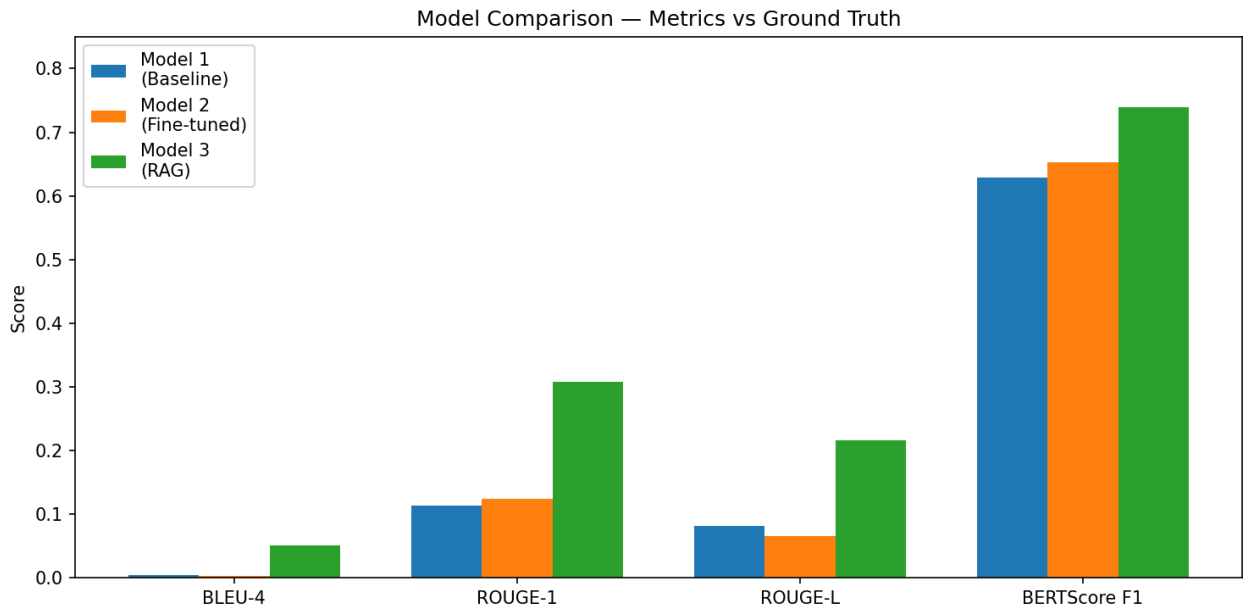

Figure 1: Metrics Comparison

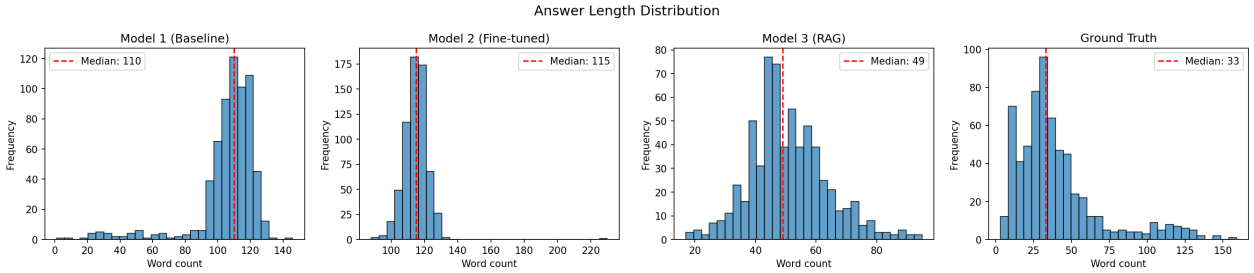

Figure 2: Answer Length Distribution

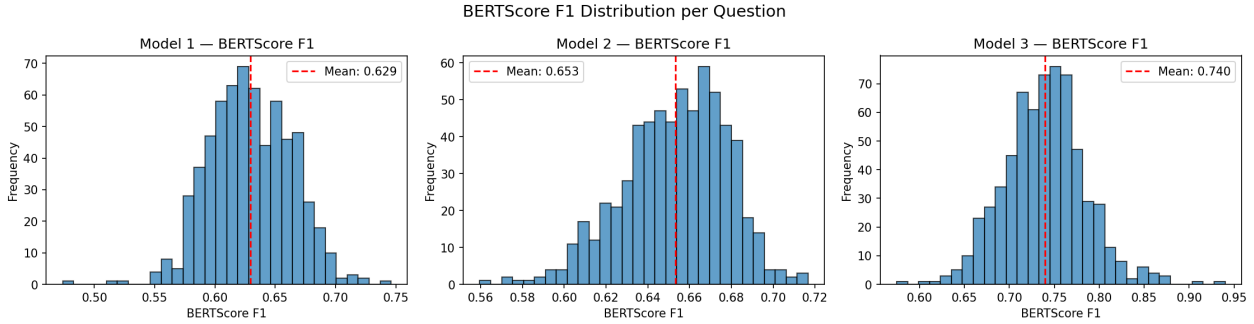

Figure 3: BERTScore F1 Distribution

Model 2 (Fine-tuned GPT-2)

- **Main issue: Confident hallucinations.** The model generates plausible-sounding legal text that is factually wrong — it invents paragraph references, cites non-existent laws, and mixes up German and Austrian legal systems.
- Example (CORP-TAX-007, asked about "Mantelkauf"): *"Der Gegenstand des Vertrages muss sich auf die Lieferung von Gegenständen beziehen... Die Gegenstände sind nach § 6 UStG steuerfrei (§ 7 Abs. 1 Z 2 EStDV)..."* — the answer has nothing to do with the question, cites the German EStDV (not Austrian law), and fabricates paragraph references.
- Many answers reference **non-existent laws** like "KstKG", "EStDV", "UStL-Vorschriften", or "KUBS", and sometimes place the context in Germany ("Wohnsitz in Deutschland", "Schleswig-Holstein") despite this being Austrian tax law.
- Fine-tuning improved topical relevance over Model 1 (answers mention tax concepts, reflected in higher ROUGE-1 and BERTScore) but not factual accuracy.

Model 3 (RAG + GPT-4o-mini)

- **Best performing model.** Answers are concise (~51 words), cite actual law paragraphs, and are factually grounded in the retrieved text.
- **Remaining issues:**
  - Retrieval misses: when the question spans multiple law areas, the top-3 retrieved chunks may not cover all relevant sections.
  - Occasionally cites the right paragraph but gives an incomplete summary.
  - Performance depends on chunk quality — some § splits break mid-sentence.

Cross-model patterns

- Models 1 and 2 share a failure mode: they generate **plausible-sounding German legal text** that is factually incorrect. This is a hallucination problem inherent to small language models without retrieval.
- Model 3 avoids this by grounding answers in retrieved law text, but is limited by retrieval quality.
- All models struggle with questions requiring reasoning across multiple law sections (e.g., interaction between KStG and EStG).

Figure 4: Error Type Distribution